

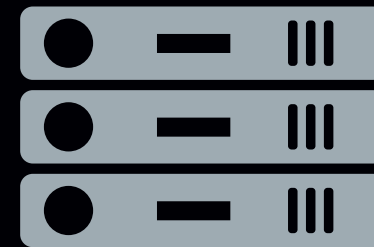


A 2 Pass Assembler & Emulator as a Service

# The Goal



**Assembler  
Emulator  
written in C99**



**POSIX Compliant  
Web Server from  
scratch in C**



**Client interface  
built with React**

# Assembler/Emulator

**C** asm.c

**C** emu.c

2 files with  
> 500 LoC?



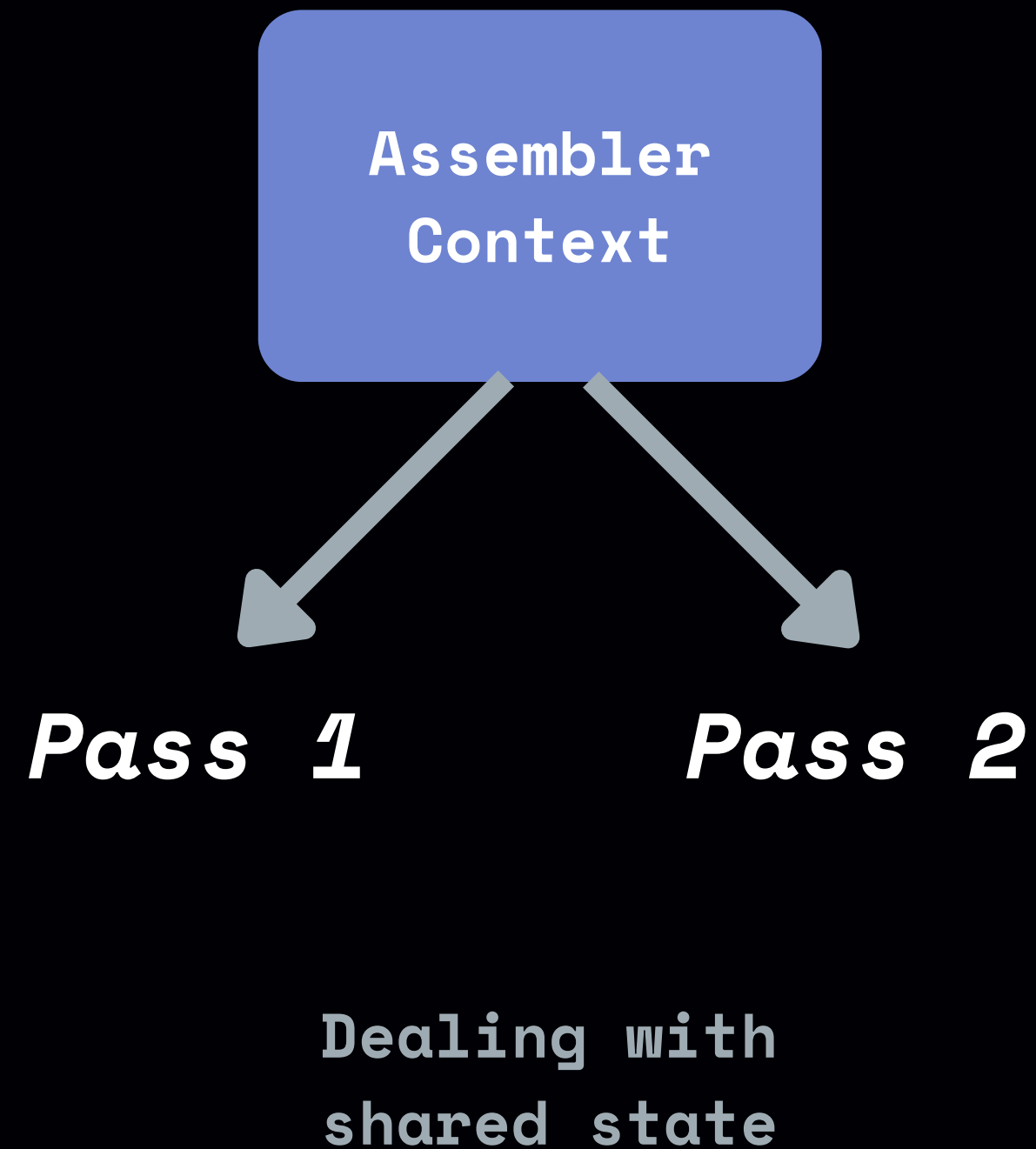
```
folder assembler
  folder build
    folder target
      file asm
      file emu
    folder include
      file assembler.h
      file emulator.h
    folder src
      folder assembler_utils
        file format.c
        file instr.c
        file pass1.c
        file pass2.c
        file symbol.c
      folder emulator_utils
        file memory.c
        file run.c
      file asm.c
      file emu.c
  folder tests
  file Makefile
```

Keeping the  
codebase modular

# Assembler/Emulator

```

└─ assembler
  └─ build
    └─ target
      ├── asm
      └─ emu
    └─ include
      ├── assembler.h
      └─ emulator.h
    └─ src
      ├── assembler_utils
      │   ├── format.c
      │   ├── instr.c
      │   ├── pass1.c
      │   ├── pass2.c
      │   └── symbol.c
      ├── emulator_utils
      │   ├── memory.c
      │   └── run.c
      ├── asm.c
      └── emu.c
  └─ tests
  └─ Makefile
```



# Assembler/Emulator

## Assembler Operating Modes

### Standard Mode

```
simplex_project/assembler on ↵ main  
› ./build/target/asm input.asm output.obj output.lst
```

### I/O Mode

```
simplex_project/assembler on ↵ main  
› echo "ldc 5 \n HALT" | ./build/target/asm -i | xxd
```

# Assembler/Emulator

## Capabilities

- Every instruction and pseudo-instruction is compatible with the assembler (including HALT, SET and data)
- The listing file shows the bytes produced for each instruction and that instruction's mnemonic at no extra I/O cost
- The first pass reads every line of the assembly code, cleans comments, saves symbols/labels to a symbol table, and recognizes mnemonics and operands, building a complete internal representation of the code (thus, not requiring any I/O with the input file in the second pass)
- The second pass uses this internal representation to correctly assign addresses to labels, and correctly translates mnemonics and appropriate opcodes into machine code and a listing file

## Errors and Warnings

- Invalid Label Name
- Illegal Immediate
- Extra on end of line
- Duplicate label
- Unknown Instruction
- Unexpected operand
- Expected an operand
- Symbol not found
- Unused Labels
- Missing HALT

# Assembler/Emulator

## Emulator Operating Modes

### Standard Mode

```
simplex_project/assembler on ↵ main  
> ./build/target/emu output.obj
```

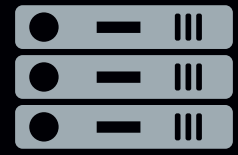
### JSON Mode

```
simplex_project/assembler on ↵ main  
> cat output.obj | ./build/target/emu -j
```

# Assembler/Emulator

## Capabilities

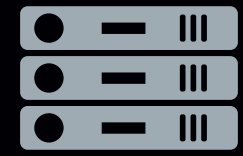
- Capable of emulating all 18 instructions of the SIMPLEX ISA
- Issues errors for invalid opcodes and potential infinite loops
- Infinite loop detection based on executing a very large number of CPU cycles (here, 100,000)
- Step by step register state and final memory trace
- Directly emulate programs in memory without requiring any writes to disk (for JSON mode)



# Web Server

*“If you wish to make an apple pie from  
scratch, you must first invent the universe”*

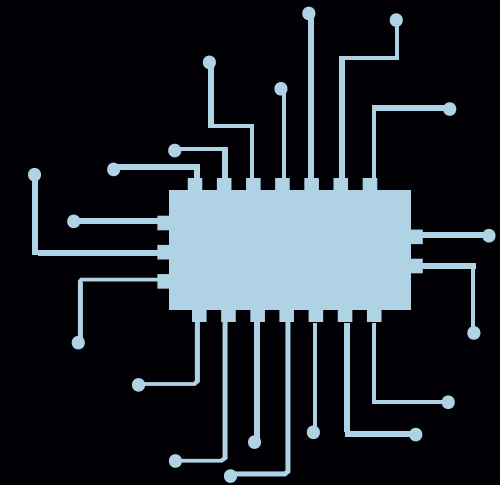
*Carl Sagan*



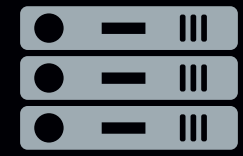
# Web Server



~10 LoC



Working with hardware



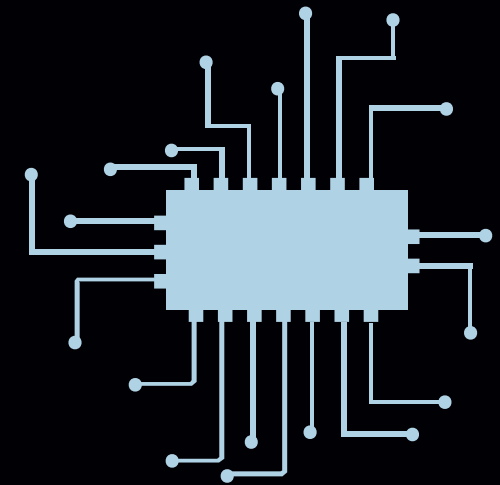
# Web Server



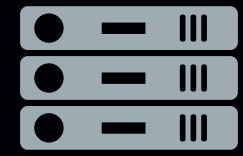
~10 LoC



Interacting directly  
with the OS (syscalls)

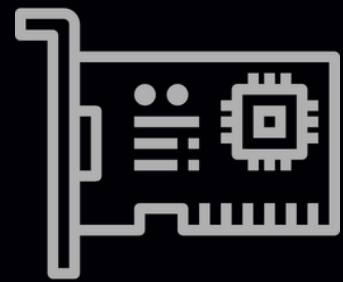


Working with hardware

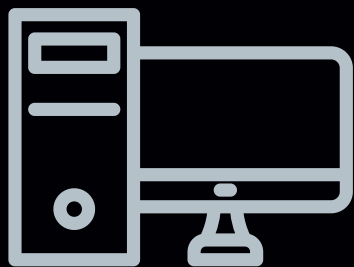


# Web Server

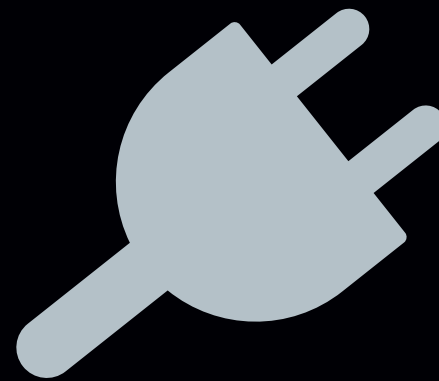
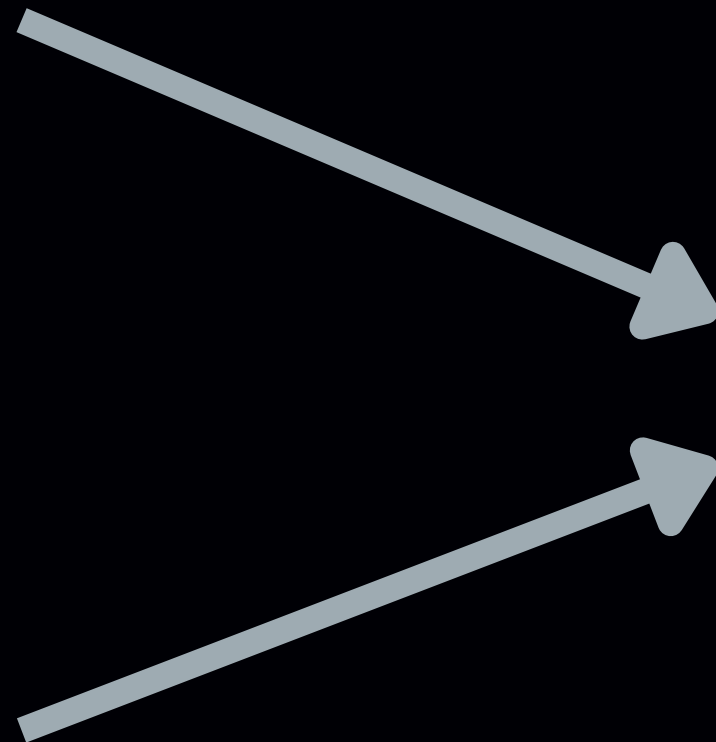
## Networking



Network  
Interface



Network  
Port

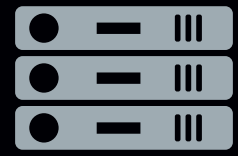


TCP Socket  
File Descriptor  
(IPv6)

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

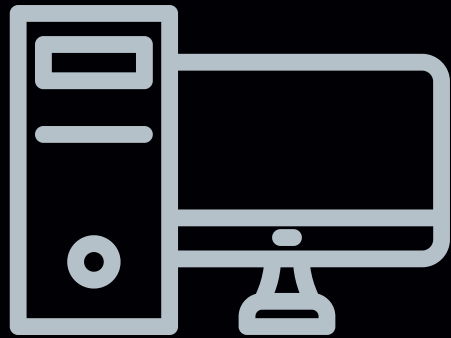
struct sockaddr_in address;
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(8080);

bind(server_fd, ... );
```



# Web Server

## Networking



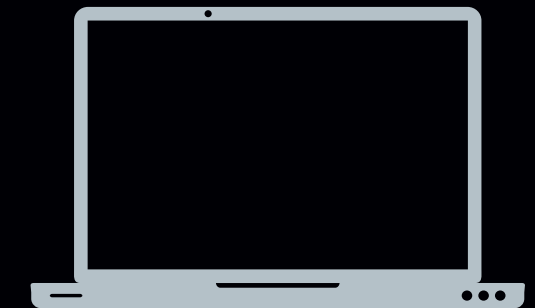
### TCP Server

Parses Packet (assume HTTP)  
Processes it  
Sends Response

010100101010100101010  
010101010010101101010

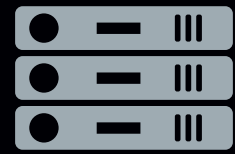


### TCP Packet



### Client

Sends request as  
TCP Packet(s)



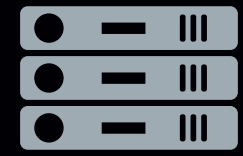
# Web Server

## Networking

```
POST /assemble HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br, zstd
Referer: http://localhost:5173/
Content-Type: text/plain; charset=UTF-8
Content-Length: 21
Origin: http://localhost:5173
Connection: keep-alive
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-site
Priority: u=0
```

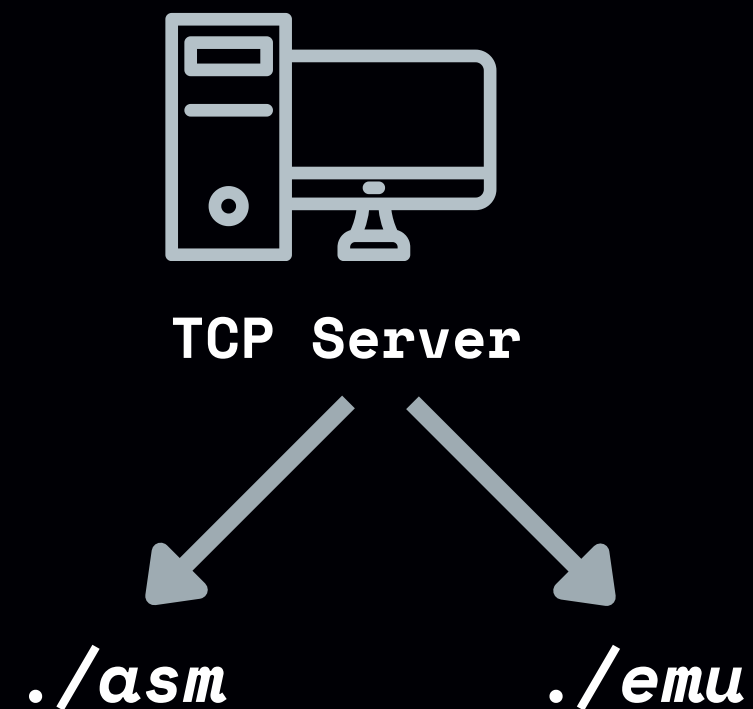
```
ldc 5
ldc 4
add
HALT
```

- **Extract HTTP Verb**
- **Extract HTTP Route**
- **Determine Content Type/Length**
- **Extract Payload**



# Web Server

How would our server talk  
to asm and emu?



Inter Process  
Communication

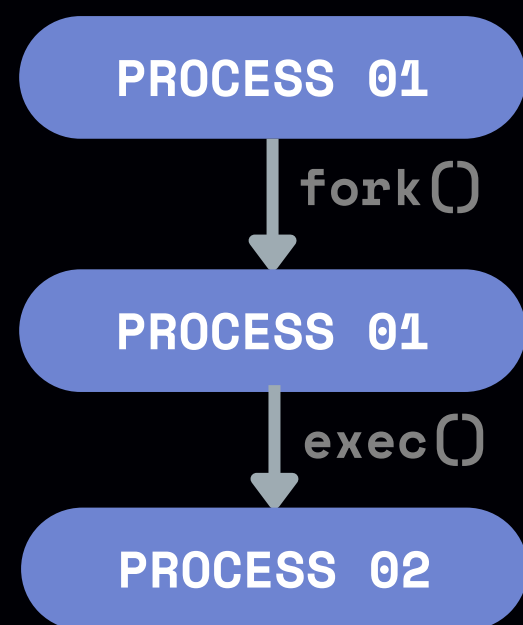


# Detour: OS Processes and Pipes

- PROCESS 01
- PROCESS 02
- PROCESS 03
- PROCESS 04

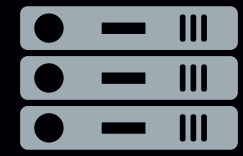
OS Process Table

- OS keeps track of every running program as a process
- Processes are run concurrently by time slicing
- Process have user level privileges
- Context switch for kernel level privileges
- Process may read/write from stdin, stdout, stderr



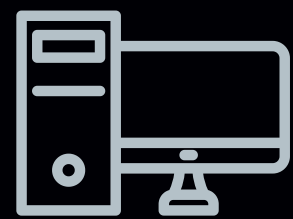
POSIX  
Pipes!





# Web Server

## Inter Process Communication



TCP Server

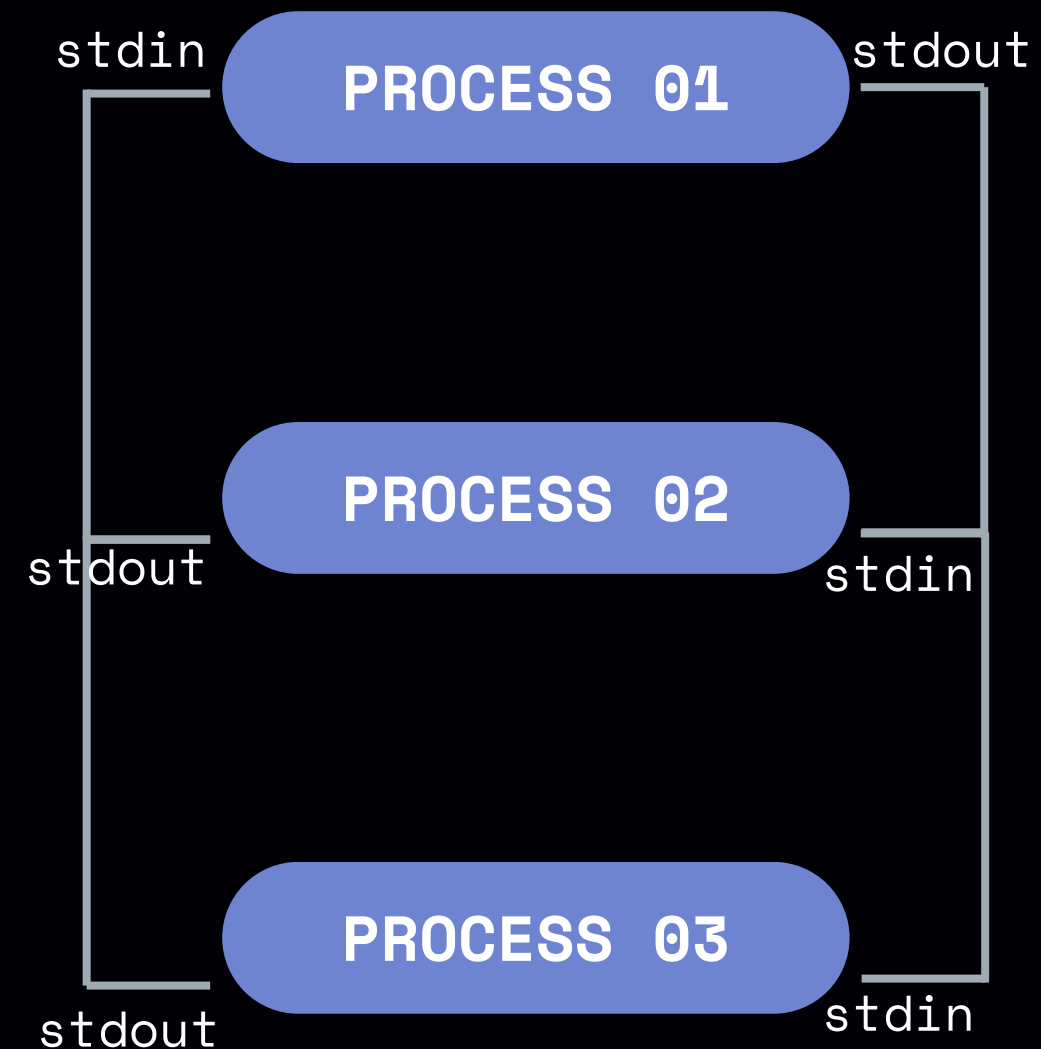
`./asm`

`./emu`

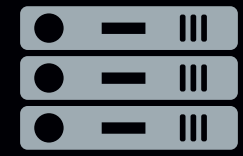
PROCESS 01

PROCESS 02

PROCESS 03



POSIX Plumbing

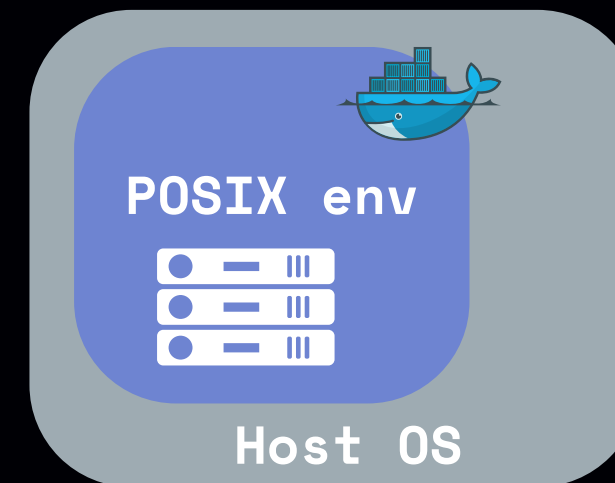


# Web Server

## A Note on POSIX

```
web_server
├── build
│   └── target
├── include
│   └── server.h
├── src
│   ├── assemble.c
│   ├── emulate.c
│   ├── main.c
│   └── router.c
├── Makefile
└── Dockerfile
```

- POSIX refers to the standard for Portable OS by IEEE
- The syscalls and pipes used are available on all POSIX Systems
- Linux, BSD, etc adhere to the POSIX standard, but Windows doesn't
- The server can be run via Docker in a non POSIX environment
- Docker allows for a minimal POSIX compliant environment to be setup using Linux cgroups





# Client Interface

## Simplex Studio

Assembly editor & emulator

### SIMPLEX ISA

Instruction Set Reference

| OP | INSTR | OPERATION                       |
|----|-------|---------------------------------|
| 0  | ldc   | B = A; A = value                |
| 1  | adc   | A = A + value                   |
| 2  | ldl   | B = A; A = memory[SP + offset]  |
| 3  | stl   | memory[SP + offset] = A; A = B  |
| 4  | ldnl  | A = memory[A + offset]          |
| 5  | stnl  | memory[A + offset] = B          |
| 6  | add   | A = A + B                       |
| 7  | sub   | A = B - A                       |
| 8  | shl   | A = B << A                      |
| 9  | shr   | A = B >> A                      |
| 10 | adj   | SP = SP + value                 |
| 11 | a2sp  | SP = A; A = B                   |
| 12 | sp2a  | B = A; A = SP                   |
| 13 | call  | B = A; A = PC; PC = PC + offset |

### Assembly Source

Write your assembly code below

```
sub ; A = N - idx
brz exit_loop

ldc idx
ldnl 0
ldc 1
add ; A = idx + 1
ldc idx
stnl 0 ; idx = idx + 1
```

```
ldc 0
brz loop
```

```
exit_loop:
HALT
```

```
; DATA
max:
data 0
idx:
data 0
arr:
data 5
data 4
data 8
data 3
data 1
data 2
N: SET 5
```

1. Assemble

2. Emulate

### Registers

Live execution state

|    |            |    |            |
|----|------------|----|------------|
| PC | 0x00000004 | SP | 0x00001FFF |
| A  | 0x00000005 | B  | 0x00000000 |

Prev

5 / 146

Next

### Memory Grid

Jump to ADDR (e.g. 64 or 0x40)

GO

| ADDR | DECIMAL    | HEX        |
|------|------------|------------|
| 27   | 4          | 0x00000004 |
| 30   | 256        | 0x00000100 |
| 31   | 6          | 0x00000006 |
| 32   | 9728       | 0x00002600 |
| 33   | 5          | 0x00000005 |
| 34   | 0          | 0x00000000 |
| 35   | 4294959887 | 0xFFFFE30F |
| 36   | 18         | 0x00000012 |

### Output

Assembler & emulator results

```
--- MACHINE CODE (HEX) ---
00 26 00 00
04 00 00 00
00 27 00 00
06 00 00 00
```

Simple UI built on:

- React
- Tailwind

ISA on the left

Step-through trace top-right

Memory Inspector on the right

Console dump bottom right

Web User Interface



# Client Interface

```
studio
├── node_modules
├── src
│   ├── components
│   │   ├── CodeEditor.jsx
│   │   ├── DebugControls.jsx
│   │   ├── ISA.jsx
│   │   ├── MemoryInspector.jsx
│   │   ├── OutputBox.jsx
│   │   └── RegisterDisplay.jsx
│   ├── lib
│   │   ├── App.jsx
│   │   ├── index.css
│   │   └── main.jsx
│   ├── index.html
│   ├── package-lock.json
│   ├── package.json
│   ├── postcss.config.js
│   ├── tailwind.config.js
│   └── vite.config.js
```

- React web app runs within a NodeJS runtime
- Dependencies managed via npm
- Vite as the build tool
- Post CSS offering Tailwind as a plugin
- Component-driven frontend for modularity

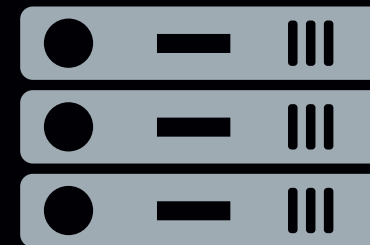
# Why?

## The Unix Philosophy



Assembler Emulator  
written in C99

Simple Modular Codebase  
Low Level Performance of C  
I/O Modes for Pipes and in-memory ops



POSIX Compliant  
Web Server from  
scratch in C

Serve responses fast with granular control  
at the OS and networking level

Assemble + Emulate in-memory for extremely  
quick responses

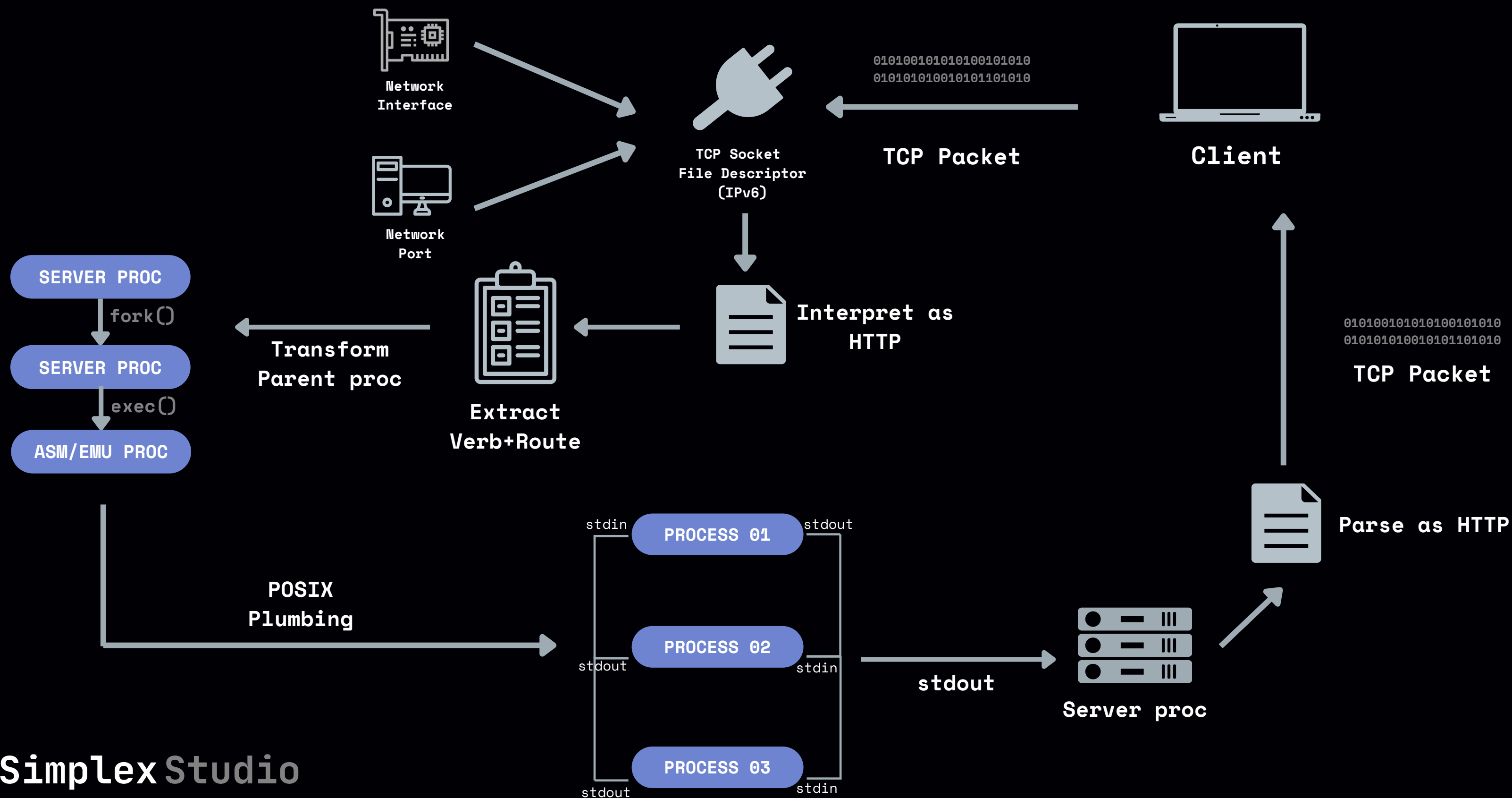
Dockerized for environment agnostic  
deployment



Client interface  
built with React

Convenient and portable interface  
Minimal Setup  
Easily Deployable

# Overview





Thank you!